Rebuilding a PingDirectory Node: Replication Came Back, the Config Didn't

> **PRIVATE DRAFT. NOT APPROVED FOR PUBLICATION.** Before making this repo public: confirm your employment agreement / client NDA allows an anonymized write-up, and resolve every **[VERIFY]** marker in this repo. If a [VERIFY] can't be answered, delete that sentence. Don't guess.

A case study from real IAM operations work: recovering one node of a four-node PingDirectory 10.3 replication topology, and then finding out that "replication is healthy" didn't mean the node was actually back.

All names, hostnames, DNs and paths in this repo are anonymized. Command examples are sanitized or marked as illustrative.

Summary

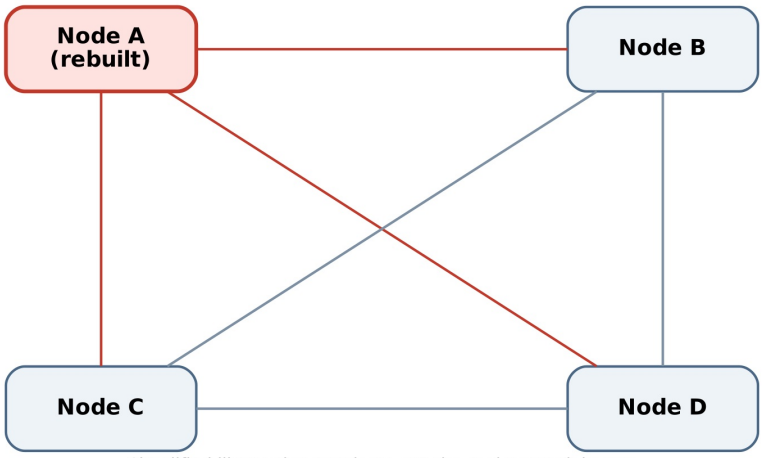
One server in a four-node PingDirectory 10.3 replication topology had to be rebuilt from scratch. Getting it back into the topology meant working through an encryption-settings mismatch, re-enabling the `userRoot` backend, re-running `dsreplication enable` and `dsreplication initialize`, and getting the server out of lockdown mode.

Once replication was confirmed healthy, I found a second problem. The rebuilt node held the same data as its peers, but it didn't behave the same. Password policy attributes that worked on the other nodes were missing on the rebuilt one, and a test user was being assigned the wrong password policy.

The cause was local server configuration, and replication never carries that. This repo covers both parts. The second part is the one I learned the most from.

Environment

- PingDirectory 10.3
- Four servers. Each one replicated with the other three.
- One `userRoot` backend, around 110k entries
- Custom password policies assigned through virtual attributes instead of being stored on each user entry
- **[VERIFY]** What used the directory (PingFederate, applications, etc.) and whether this was a production or non-production tier. Don't call it production unless it was.



Simplified illustrative topology: not the real network layout
(each line = two-way replication)

Simplified, illustrative topology. Not a copy of the real network layout.

What happened, in short

1. **Node A had to be rebuilt. [VERIFY: why, what the first symptom was, and who or what noticed it.]**
2. **Getting it back into replication** took four things: fixing an encryption-settings mismatch, re-enabling `userRoot`, running `dsreplication enable` + `initialize` from a healthy peer, and clearing lockdown mode.
3. **Replication checks passed.** 3 peers, 3 in / 3 out connections, backlog 0, 110,885 entries, 0 failed replication operations.
4. **But Node A behaved differently.** The same LDAP query against the same test user gave different answers on Node A and Node B.
5. **Cause: config drift.** A virtual attribute and a plugin were disabled on the rebuilt node, and three custom virtual attributes that assign password policies were missing. Replication copies data, not server configuration, so none of this came back with `initialize`.
6. **Fix:** targeted `dsconfig` changes to match the healthy peer, validated one at a time. Then a planned config-parity pass in small groups instead of applying the full config diff.

Key takeaway

A node with correct data and the wrong configuration passes every replication check. When I bring a directory server back now, "recovered" means two things: the data matches, **and** the server behaves the same as its peers.

What these numbers do and don't mean

- **110,885 entries**: the `userRoot` entry count on the recovered node after initialization.
- **0 failed**: the replication failed-operations counter on Node A and Node B. It means no replicated changes failed to apply. It does **not** mean no client LDAP writes failed during the incident. This repo doesn't claim zero downtime or zero client impact.
- **[VERIFY]** Entry counts and failed-op counters on Node C and Node D.

Command examples in this repo are either sanitized versions of commands I ran (hostnames, DNs and paths replaced) or clearly marked as illustrative. Check exact syntax against the documentation for your PingDirectory version.

Investigation

This covers what I saw, what I checked, and how I got to each cause. The actual fixes and validation are in Recovery and validation.

Part 1: the failed node

What happened

Node A needed a full rebuild. **[VERIFY: why. Failed upgrade? Corrupt backend? Disk? A change gone wrong?]**

[VERIFY: what was the first symptom, and who or what noticed it (an alert, a user report, a failed job)?]

[VERIFY: did the other three nodes keep replicating with each other and serving traffic while Node A was out?]

[VERIFY: one or two sentences on why you rebuilt instead of repairing in place.]

The encryption-settings mismatch

PingDirectory can encrypt data at rest using encryption-settings definitions. In a replicated topology that uses this, the servers need to share the same definitions. A server that doesn't have the right definition can't work with data that was encrypted using it. **[VERIFY this explanation against the Ping 10.3 docs before publishing.]**

The rebuilt Node A didn't match its peers, so this had to be fixed before replication could work.

[VERIFY: how you noticed the mismatch. An error message? `encryption-settings list` output compared across nodes? Only add commands you actually ran.]

Lockdown mode

After initialization, Node A came up in lockdown mode. In lockdown, PingDirectory only accepts requests from administrators. So the server can be running and still clearly not be ready for normal traffic.

[VERIFY: what caused it and what you checked to find out.]

Part 2: the config drift

This is the part that wasn't obvious. Replication on Node A was already confirmed healthy (see Recovery and validation).

The symptom

I ran the same query against the same test user on Node A and Node B. I asked for `ds-pwp-state-json`, a virtual attribute that shows a user's password policy state as JSON.

Node B returned it. Node A returned nothing.

```
# Sanitized. Real hostnames, DNs and paths replaced.
bin/ldapsearch \
  --hostname node-a.example.com --port 1636 --useSSL \
  --bindDN "cn=admin-user" \
  --bindPasswordFile /path/to/bindPasswordFile \
  --baseDN "cn=test-user,ou=people,dc=example,dc=com" \
  --searchScope base \
  "(objectClass=*)" ds-pwp-state-json
```

Same data, different answer. That told me the problem wasn't in the data. It was in how the server was configured to process it.

A small thing that slowed me down: one of the test entries had an RDN ending in an escaped trailing space. Copying the DN by hand kept breaking my queries. I switched to capturing DNs straight from `ldapsearch --dontWrap` output with command substitution, so the exact DN went into the next command untouched.

Finding 1: a virtual attribute and a plugin were disabled

I compared the relevant server configuration on both nodes:

Config object	Node A	Node B
Password Policy State JSON (virtual attribute)	disabled	enabled
Modifiable Password Policy State (plugin)	disabled	enabled

That explained why `ds-pwp-state-json` was missing on Node A.

Finding 2: users were getting the wrong password policy

This one was more serious. On Node B, the test user resolved to a custom password policy. On Node A, the same user fell back to the default policy.

In this environment, password policies weren't stored on each user. They were assigned by three custom virtual attributes, each one setting `ds-pwp-password-policy-dn` for a different group of entries. Node B had all three. Node A had none of them.

If Node A had taken real bind traffic in that state, users who landed on it would have been checked against the default policy instead of their real one. That means different lockout and password rules depending on which server they happened to hit.

Root cause

Replication copies directory data between servers. It doesn't copy each server's own configuration. That lives separately on every server.

When Node A was rebuilt, it started with default configuration. Anything that had been customized on the old Node A over time, like enabled plugins and custom virtual attributes, was gone unless someone added it back. `dsreplication initialize` brought the data back. It couldn't bring back the config, because config was never part of replication.

So every replication health check passed, and the node still behaved differently from the others.

Confirmed vs. not confirmed:

- **Confirmed:** the specific config differences above, and that fixing them made Node A return the same results as Node B.
- **Not confirmed:** whether these differences existed on the old Node A before the rebuild, or only appeared because of it. Without a config export from before the rebuild, there's no way to prove which. "Lost in the rebuild" is the most likely explanation, not a proven one.

Recovery and validation

The steps below are in the order I did them. For each one: what I did, why, and how I checked it before moving on.

Part 1: getting Node A back into replication

1. Fix the encryption settings

What: brought Node A's encryption-settings definitions in line with its peers.

Why first: without matching definitions, the rebuilt node couldn't work with encrypted data from the rest of the topology. There was no point initializing until this was fixed.

[VERIFY: exactly how you fixed it. For example, exported the definition from a healthy node and imported it on Node A? Only add commands you actually ran.]

2. Re-enable the `userRoot` backend

What: the `userRoot` backend on Node A was disabled and had to be re-enabled before replication could be set up for it. **[VERIFY: why it was disabled, if you know.]**

```
# Illustrative. Confirm the exact command you used.
bin/dsconfig set-backend-prop \
  --backend-name userRoot \
  --set enabled:true
```

3. `dsreplication enable`, then `dsreplication initialize`

What: added Node A back into the topology with `dsreplication enable`, then ran `dsreplication initialize` to load its data from a healthy peer.

Risk I watched for: `initialize` replaces the target server's data with the source server's data. Running it in the wrong direction would have pushed Node A's empty or incomplete data onto a good node. Before running it, I made sure the source was a healthy node and the target was Node A. **[VERIFY: which node you used as the source.]**

4. Clear lockdown mode

What: after initialization, Node A was in lockdown mode, so it wasn't ready for normal traffic.

[VERIFY: how you cleared it. `bin/leave-lockdown-mode`? A restart? Fixing something first?]

5. Confirming replication health

Working with Ping support, we confirmed Node A's replication state:

Check	Result
Configured replication peers	3
Replication connections	3 inbound, 3 outbound
Replication backlog	0
`userRoot` entry count	110,885
Failed replication operations	0 (Node A and Node B)

Scope of these numbers: "0 failed" is the replication counter. It means no replicated changes failed to apply. It does **not** prove that no client LDAP writes failed during the incident, and I'm not claiming that.

[VERIFY: entry counts and failed-op counters on Node C and Node D. Keep the `dsreplication status` / `status` output as evidence.]

By the usual checks, Node A was recovered. It wasn't, quite. See Investigation, Part 2.

Part 2: fixing the config drift

6. Enable the password policy state virtual attribute and plugin

What: enabled the Password Policy State JSON virtual attribute and the Modifiable Password Policy State plugin on Node A, with the same base DN scope and filter settings as Node B.

```
# Illustrative. Check exact object names with list-virtual-attributes / list-plugins.
bin/dsconfig set-virtual-attribute-prop \
  --name "Password Policy State JSON" \
  --set enabled:true

bin/dsconfig set-plugin-prop \
  --plugin-name "Modifiable Password Policy State" \
  --set enabled:true
```

Validation: re-ran the same `ldapsearch` against the same test user. `ds-pwp-state-json` was now returned on Node A.

7. Recreate the password-policy virtual attributes

What: recreated the three custom virtual attributes on Node A to match Node B exactly, including their `evaluation-order-index` values (10, 2 and 2).

Why the order index mattered: it controls which rule applies when more than one could match an entry. Copying the attributes without the same order could have assigned the wrong policy to some users, which is the exact problem I was fixing.

Validation: the test user resolved to the correct custom policy on Node A, same as on Node B.

Part 3: why I didn't apply the full config diff

I generated a configuration comparison between Node A and Node B. The obvious move was to take every difference and apply it to Node A.

Ping support advised against that, and once I read through the diff, I understood why. It mixed shared settings with settings that are *supposed* to be different on each server: certificates and keystores, trust settings, debug logging. Applying it wholesale could have broken Node A's TLS setup or copied Node B's server-specific settings onto it.

So the plan became:

1. Split the shared differences into functional groups (password validators, virtual attributes, password policy settings, and so on).
2. Take a config snapshot before each group.
3. Apply one group, then validate it with real queries before moving on.
4. Leave node-specific values alone.
5. Handle index differences last, as their own separate change.

[VERIFY: current status of the config-parity work. Is it finished? If yes, how did you confirm parity at the end?]

Lessons learned

What I'd do differently

Capture config before rebuilding. Before tearing down a node, I'd export its configuration so there's a known baseline to compare against afterward. Diffing the rebuilt node against a peer works, but a before/after of the same node is much cleaner. It would also have answered the one question I couldn't: whether the drift was caused by the rebuild or was already there.

Don't stop at "replication is healthy." Backlog 0 and matching entry counts prove the data is in sync. They say nothing about how each server processes that data. Next time I'd add a behavior check: the same queries against the same test users on every node, comparing password policy state and effective policy.

Keep custom config as code. Custom virtual attributes and plugin settings should live in version-controlled `dsconfig` batch files. Then a rebuilt node gets the same config by running a script, instead of depending on someone remembering what was changed by hand.

Use a bind password file from the first command. Passwords typed on the command line end up in shell history. `--bindPasswordFile` should be the default, not something you switch to halfway through.

Be sure of the direction before running `initialize`. It overwrites the target. Write down the source and target before pressing enter.

A post-rebuild checklist I'd use now

- [ ] Encryption-settings definitions match the rest of the topology
- [ ] Backends enabled as expected
- [ ] `dsreplication status`: all peers connected, backlog 0
- [ ] Entry counts match on **every** node, not just the recovered one
- [ ] Server not in lockdown mode
- [ ] Same test queries return the same results on every node (including password policy state and effective policy)
- [ ] Config compared against a healthy peer, with node-specific settings (certs, keystores, trust, logging) excluded
- [ ] Shared config differences applied in small groups, each validated before the next

Closing

The replication recovery was the part I expected to be hard, and it was: encryption settings, a disabled backend, lockdown mode. But the issue that would have actually affected users was the quiet one afterward. A node with correct data and the wrong configuration passes every replication check.